Laporan Praktikum MLOps: Hands-on 1 — MLflow Experiment Tracking & Model Registry

Identitas	Keterangan
Nama Praktikan	Givaro Ananta
NIM	123450078
Program Studi	Sains Data
Mata Kuliah	Machine Learning Operations (MLOps) — Hands-on 1

---

1. Pendahuluan & Tujuan Praktikum

Praktikum **Hands-on 1** berfokus pada penguasaan modul dasar dan arsitektur inti dari **MLflow**, mencakup:

1. Menjalankan **MLflow Tracking Server** dengan basis data SQLite dan penyimpanan artefak lokal.
2. Melakukan **Logging Dasar** pada model *machine learning* tradisional (scikit-learn).
3. Melakukan **Logging per Epoch** pada arsitektur *deep learning* (PyTorch) lengkap dengan pelacakan *learning rate*, *checkpointing*, dan *data hash*.
4. Memanfaatkan fitur **Autologging** untuk otomatisasi pencatatan artefak dan metrik.
5. Melakukan **Hyperparameter Tuning** terstruktur menggunakan **Optuna** dengan konsep **Nested Runs** (Parent Run dan Child Runs).
6. Melakukan pencarian run secara programatik (`mlflow.search_runs`) serta pengelolaan siklus hidup model pada **MLflow Model Registry** dengan sistem *aliasing* (`@challenger` dan `@champion`).

---

2. Arsitektur & Lingkungan Tracking Server (Slide 41)

Sesuai spesifikasi pada Slide 41, server pelacak dijalankan dengan konfigurasi:

- **Backend Store URI:** `sqlite:///mlflow.db` (menyimpan metadata runs, parameters, metrics, dan tags).
- **Default Artifact Root:** `./mlartifacts` (menyimpan berkas model, checkpoint, dan visualisasi).
- **Endpoint Server:** `http://0.0.0.0:5000` (dapat diakses lokal via `http://127.0.0.1:5000`).

Skrip peluncuran diwujudkan dalam `start_server.sh`:

```
mlflow server --backend-store-uri sqlite:///mlflow.db \
              --default-artifact-root ./mlartifacts --host 0.0.0.0 --port 5000
```

Server berhasil aktif dan mengembalikan respons `HTTP 200 OK`.

---

3. Hasil Percobaan per Modul

3.1. Modul 1: Logging Dasar scikit-learn (Slide 42 & 43)

- **Berkas:** `scripts/01_logging_dasar.py`
- **Dataset:** Breast Cancer Wisconsin (`load_breast_cancer`, 569 sampel, 30 fitur).
- **Model:** `RandomForestClassifier(n_estimators=200, max_depth=8, random_state=42)`.
- **Identitas Eksperimen:**
  - **Experiment Name:** `breast-cancer-clf`
  - **Run Name:** `rf-baseline`
  - **Author Tag:** `Givaro Ananta` (NIM: `123450078`)
- **Hasil Eksekusi:**
  - **Run ID:** `21e232098d8a49e89f2ff97293c17cf7`
  - **Val F1 Score:** `0.9655`
  - **Val ROC-AUC:** `0.9940`
  - **Model Artifact:** Berhasil disimpan ke MLflow Models flavor `sklearn` dengan skema `skops_trusted_types`.

3.2. Modul 2: Deep Learning — Logging per Epoch dengan PyTorch (Slide 44 & 45)

- **Berkas:** `scripts/02_pytorch_logging_per_epoch.py`
- **Arsitektur:** ResNet-18 (`cfg = {"lr": 3e-4, "batch_size": 64, "epochs": 20, "arch": "resnet18", "seed": 42}`).
- **Data Lineage:** Hash dataset dihitung menggunakan MD5 dan dicatat pada tag/parameter `data_md5`.
- **Environment Lineage:** Berkas `requirements.txt` dilog sebagai artefak penunjang reproduktifitas.
- **Proses Pelatihan (20 Epoch):**
  - Setiap epoch mencatat `train_loss`, `val_loss`, `val_f1`, dan `lr` secara berurutan dengan parameter `step=epoch`.
  - Mekanisme *checkpointing* otomatis menyimpan bobot `best.pt` setiap kali `val_f1` memecahkan rekor terbaik.
- **Hasil Akhir:**
  - **Run ID:** `968a74a7df4443d7975974a2ea45ec4d`
  - **Best Val F1:** `0.0518` (tercatat pada Epoch 7)
  - **Train Loss:** Menurun dari `2.3447` menjadi `2.1910`
  - **Model Flavor:** `mlflow.pytorch.log_model` berhasil mencatat model secara terisolasi.

3.3. Modul 3: Autologging (Slide 46)

- **Berkas:** `scripts/03_autologging.py`
- **Fitur:** Mengaktifkan `mlflow.autolog()` dan `mlflow.sklearn.autolog()`.
- **Kelebihan Teramati:**
  - MLflow secara otomatis menangkap seluruh hyperparameter (`n_estimators`, `max_depth`, `criterion`, dll.).

  - Kurva evaluasi dan matriks konfusi digenerate dan disimpan ke dalam artefak tanpa satu pun baris kode logging manual.
- **Praktik Terbaik:**
  - Tetap menyematkan tag konteks bisnis dan identitas praktikan (**Givaro Ananta - 123450078**) secara eksplisit.
- **Hasil:**
  - **Run ID:** `07083129b745435b8a5fac30ad8d5ccb`
  - **Akurasi Validasi:** `0.9474` (94.74%)

3.4. Modul 4: Hyperparameter Tuning dengan Nested Runs (Slide 47 & 48)

- **Berkas:** `scripts/04_optuna_nested_runs.py`
- **Framework: Optuna + MLflow Tracking**.
- **Ruang Pencarian Hyperparameter:**
  - `lr`: $[1 \times 10^{-5}, 1 \times 10^{-2}]$ (distribusi logaritmik)
  - `batch_size`: $[32, 64, 128]$
  - `dropout`: $[0.0, 0.5]$
- **Struktur Hierarki Runs:**
  - **Parent Run:** `optuna-sweep-01` (Run ID: `e831a60f14e6430b836811ee487a2b7d`)
  - **Child Runs:** 20 Trial (`trial-0` hingga `trial-19`) dengan penanda `nested=True`.
- **Hasil Tuning:**
  - **Best Trial:** Trial #10
  - **Best Val F1:** `0.9719`
  - **Best Hyperparameters:**
    - `lr`: `0.0079965`
    - `batch_size`: `32`
    - `dropout`: `0.454975`
- **Keunggulan UI:** Pada MLflow UI, seluruh 20 child run tersusun rapi di bawah parent run dan dapat dianalisis menggunakan *Parallel Coordinates Plot* untuk melihat korelasi antar parameter terhadap skor `val_f1`.

3.5. Modul 5: Programmatic Search & Model Registry (Slide 49 & 50)

- **Berkas:** `scripts/05_search_and_registry.py`
- **Pencarian Run Programatik (Slide 49):**
  - Filter query:

```
filter_string="metrics.val_f1 > 0.95 and params.max_depth = '8' and tags.stage != 'deb
```

  - Query berhasil menemukan Run ID terbaik: `21e232098d8a49e89f2ff97293c17cf7` dengan `val_f1 = 0.9655` dan `val_auc = 0.9940`.
- **Model Registry Lifecycle (Slide 50):**
  1. **Registrasi:** Model dari run terbaik didaftarkan dengan nama `breast-cancer-clf` (menghasilkan Versi 1).

2. **Anotasi & Tagging:** Diberikan deskripsi dan metadata identitas `Givaro Ananta (123450078)` serta tag `validation_status: passed`.
3. **Aliasing Awal:** Diberi alias `challenger`.
4. **Promosi:** Setelah lolos pengujian, alias dipromosikan menjadi **`champion`**.
5. **Model Serving & Loading:** Model dimuat secara dinamis melalui URI alias:

```
model = mlflow.pyfunc.load_model("models:/breast-cancer-clf@champion")
preds = model.predict(X_new)
```

– **Hasil Prediksi:** Model berhasil melakukan inferensi pada 5 sampel uji dengan prediksi: `[0, 1, 0, 0, 0]`.

---

4. Eksplorasi Visual & Analisis pada MLflow UI (http://localhost:5000)

Pada antarmuka Web UI MLflow (`http://localhost:5000`), evaluasi mendalam dilakukan dengan memanfaatkan fitur visualisasi terintegrasi:

4.1. Membandingkan Multi-Run (Compare Runs)

Dengan memilih beberapa run sekaligus lalu menekan tombol **Compare**, praktikan dapat menganalisis perbandingan performa lintas eksperimen:

- **Parallel Coordinates Plot:** Digunakan pada 20 *child runs* Optuna (Modul 04) untuk memetakan alur kombinasi hyperparameter multi-dimensi (`lr`, `batch_size`, `dropout`) terhadap target `val_f1`. Visualisasi ini menunjukkan dengan jelas bahwa kombinasi *learning rate* di rentang $0.007 - 0.008$ bersama *batch size* 32 mengerucut pada F1-score optimal (0.9719).
- **Scatter Plot & Contour Plot:** Memetakan relasi berpasangan antara parameter kontinu (misalnya `dropout` vs `val_f1`) serta memvisualisasikan gradien permukaan kontur performa, sehingga area hyperparameter terbaik dapat diidentifikasi secara intuitif tanpa membaca tabel metrik numerik satu per satu.

4.2. Overlay Learning Curves Antar Run

Pada eksperimen deep learning PyTorch (Modul 02), fitur **Overlay Learning Curves** memungkinkan penumpukan kurva metrik per-epoch (`train_loss`, `val_loss`, dan `val_f1`) secara berdampingan antar iterasi run. Fitur ini sangat efektif untuk memvalidasi kestabilan pelatihan secara visual, mendeteksi terjadinya *overfitting* (ketika *train loss* terus turun sementara *val loss* mulai mendaki), serta memastikan konvergensi model berjalan dengan mulus.

4.3. Inspeksi Artefak (Model, Plot) dan Diff Parameter

- **Pemeriksaan Artefak:** Di dalam tab *Artifacts* pada setiap run, MLflow menampilkan pohon berkas lengkap yang mencakup serialisasi bobot model (`model/`), berkas dependensi lingkungan (`requirements.txt`, `conda.yaml`, `MLmodel`), checkpoint PyTorch `best.pt`, serta grafik matriks konfusi hasil autologging.
- **Diff Parameter:** Fitur pembanding parameter menyoroti secara langsung perbedaan nilai konfigurasi antar dua atau lebih run (misal membandingkan nilai `n_estimators`, `max_depth`,

atau arsitektur jaringan), mengeliminasi risiko ketidaktelitian manusia saat melakukan audit konfigurasi model sebelum dipromosikan ke tahap **Model Registry** di bawah alias `@champion`.

---

5. Analisis & Refleksi Prinsip MLOps (Slide 51 & 52)

Penerapan Hands-on 1 ini membuktikan konsep penting dalam arsitektur MLOps:

1. **Decoupling (Pemisahan Tanggung Jawab):**
   - Aplikasi serving tidak perlu mengetahui path fisik file model maupun nomor versinya secara hardcoded. Cukup memanggil `models:/<nama_model>@champion`.
2. **Reproducibility & Lineage:**
   - Setiap run melacak data hash (`data_md5`), parameter arsitektur, dan berkas dependensi (`requirements.txt`).
3. **Maturity Level:**
   - Implementasi ini menaikkan kematangan sistem dari **Level 0 (Manual / Notebook)** menuju **Level 1 (ML Pipeline Automation)**, di mana seluruh metadata dan artefak tersimpan secara terpusat dan dapat diaudit secara programatik.

---

6. Kesimpulan

1. Seluruh kode Hands-on 1 pada Slide 39–52 berhasil dijalankan 100%.
2. Lingkungan MLflow Tracking Server dan Model Registry bekerja secara stabil pada port 5000 dengan basis data SQLite.